Chapter-4

(How instructions are being executed inside processor?)

$$\text{CPU time} = \underbrace{\text{Instruction count}}_{\text{Determined by the language or platform you are using.}} \times \underbrace{\text{CPI} \times \text{Clock Cycle Time}}_{\text{Determined by CPU hardware.}}$$

language or platform → ISA, Compiler

→ How many Risc-V instructions we need to complete the task/code?

* Two types of Risc-V implementations –

(i) A simplified version (Slow, too much time wasted)

(ii) A more realistic pipelined version (Optimized)

* We will only look into subset of instructions divided into 3 categories –

(i) Memory Reference : ld, sd → (main memory associated)

(ii) Arithmetic / Logical : add, sub, and, or

(iii) Control Transfer : beq → (Branching)

Instruction Execution :

Instruction Memory	Data Memory

(holds the instructions only)

(Refers to the main-memory)
e.g ⇒ while using load / store instruction, this memory is accessed.

Instruction Execution

- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch comparison
  - Access data memory for load/store
  - PC ← target address or PC + 4

#8000

PC contains the address of the instruction in the program being executed.

Now, go to that address in the and fetch that instruction

Instruction Memory

#8000 → add X_{20}, X_{21}, X_0

the fetched instruction will now be decoded.

↳ the registers associated with that decoded instruction will be activated to read data.

Sample Simulation of the ins. exe. Steps

PC = $\boxed{\#8000}$

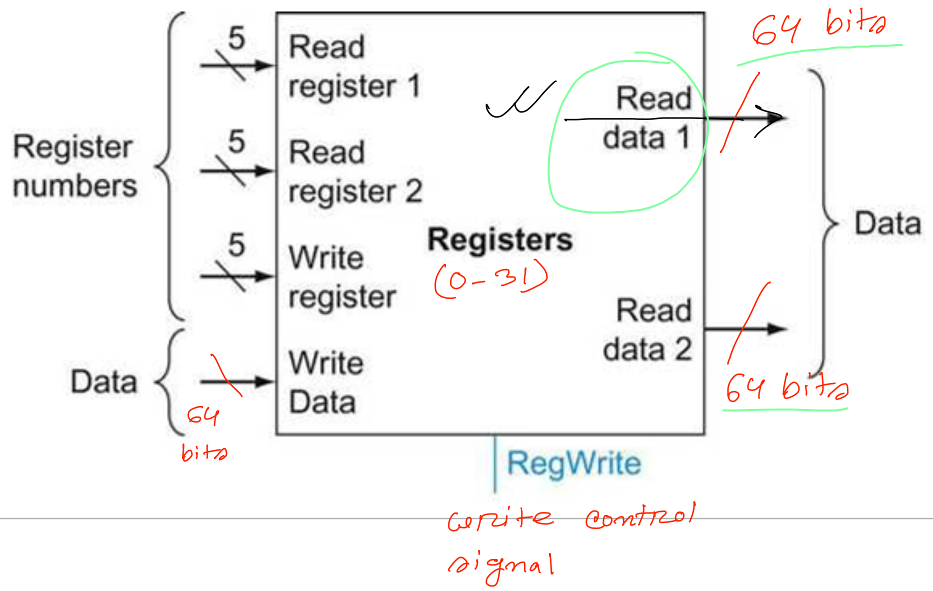
Add x6, x20, x5

ins. Memory

#8000	xxxx xxx0
#8001	0101 1010
#8002	0xxx 0011
#8003	0xxx xxxx

$$\underbrace{xxxx\ xxx}_{f7}\underbrace{0\ 0101}_{rs2}\ \underbrace{1010\ 0}_{rs1}\underbrace{xxx}_{f3}\ \underbrace{0011\ 0}_{rd}\underbrace{xxx\ xxxx}_{op\text{-}code}$$

Register File



(i) Read ⟹ Provide the register number that you want to read through the Read register bus.

⟹ Data output will come from Read data bus.

(ii) Write ⟹ Provide the register number that you want to write through the Write register bus.

⟹ Provide the data to be written via the Write data bus.

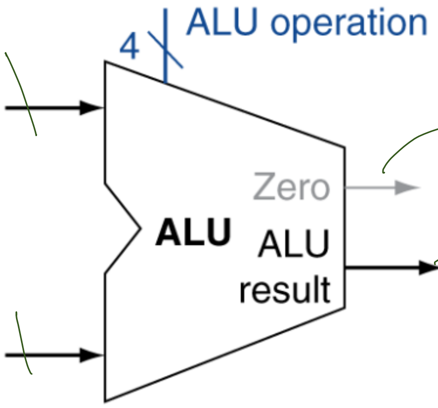
⟹ Write control signal must be asserted.

regWrite

ALU (Arithmetic Logic Unit)

tells ALU which operation to perform on the inputs.

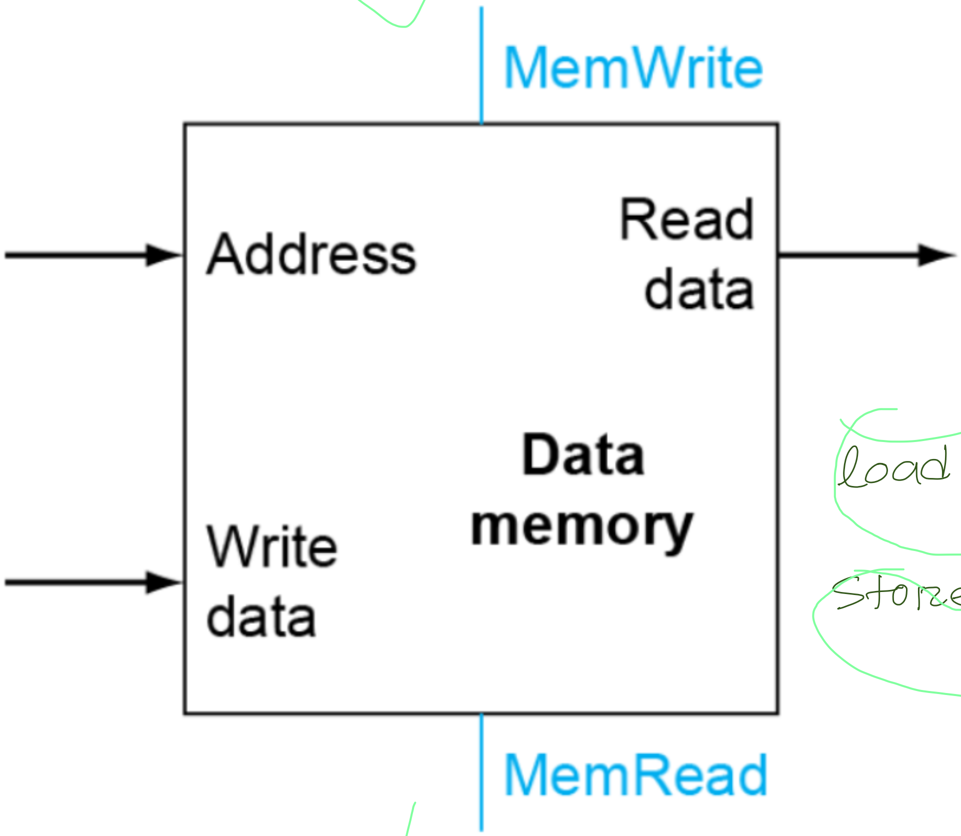
size of both inputs must be same



special output pin that tells if the ALU result is zero or not. Used in Branch calculation.

Actual result.

Data Memory

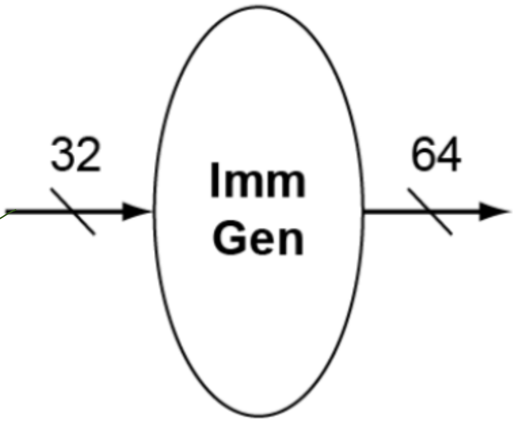


load => read data

store => write data

Immediate Generation unit:

↳ Expands 12 bit immediate to 64 bit value.



⇒ ld x21, 10(x22)

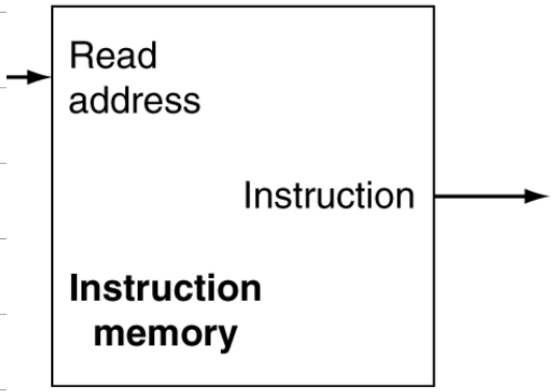
Size of immediate in I type instruction is 12 bits.

⇒ Size of the complete instruction is 32 bits.

⇒ which is entered as input in the immediate generation unit.

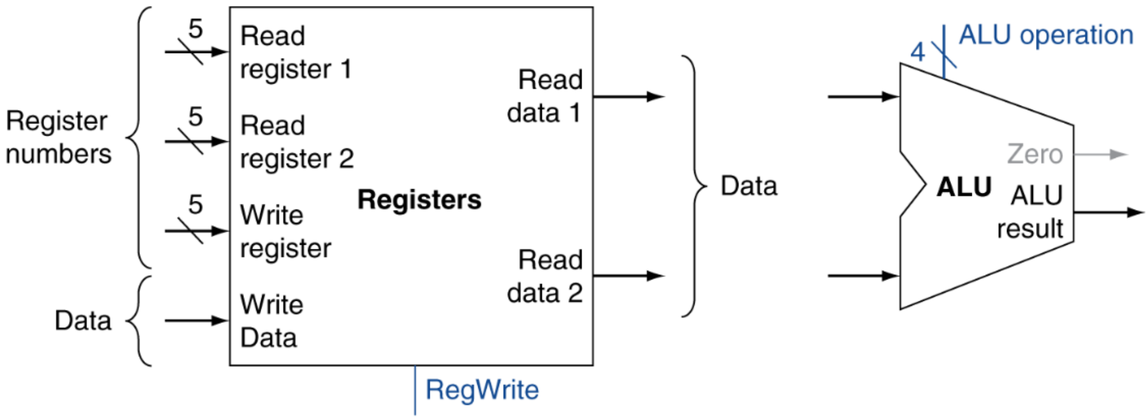
⇒ This unit then extracts the 12 bit immediate and expands it into 64 bit.

Instruction Memory



R-Format Instructions

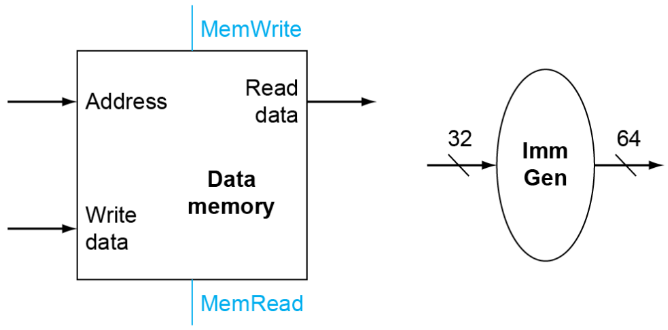
- Read two register operands
- Perform arithmetic/logical operation
- Write register result



a. Registers b. ALU

Load/Store Instructions

- Read register operands
- Calculate address using 12-bit offset
  - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit

b. Immediate generation unit

Branch Instructions